
Deep Reinforcement Learning for Compiler Optimization

Pass Ordering in LLVM

Anonymous Author(s)

Affiliation

Address

email

Abstract

The phase ordering problem—determining the optimal sequence of compiler optimization passes—is NP-hard and has resisted efficient exact solutions for decades. We present a deep reinforcement learning (DRL) system that learns pass-ordering policies for the LLVM compiler using the CompilerGym framework Cummins et al. [2022], without relying on large language models or hand-crafted heuristics. Our approach combines recurrent Proximal Policy Optimization (PPO) Schulman et al. [2017] with three key contributions: (1) a carefully designed state representation that augments Autophase IR features Huang et al. [2019] with per-feature online normalization and per-action usage counts; (2) truncation-aware Generalized Advantage Estimation (GAE) Schulman et al. [2015] that correctly bootstraps value targets at episode boundaries; and (3) an inference-time ensemble of independently trained policies with best-of- k stochastic rollouts that exploits diversity across training runs at zero additional training cost. Evaluated on `cbench-v1`, our best single model achieves a mean reward of 1.044 and beats `-Oz` on 17 out of 23 programs, while the final seven-model ensemble reaches 1.050 and beats `-Oz` on 20 of 23 programs—doubling the count compared to our PPO baseline. Cross-suite evaluation on six additional CompilerGym benchmark suites demonstrates that the ensemble generalizes zero-shot, outperforming random and single-model baselines on every suite tested.

1 Introduction

The phase ordering problem is a fundamental challenge in compiler optimization: it asks for the optimal ordering of compilation passes such that the resulting low-level code achieves the best performance or the smallest code size. Different pass orderings can produce substantially different machine code, making the choice of optimization schedule particularly consequential in performance-critical settings. Despite decades of research, the problem remains NP-hard in the general case, and no efficient algorithm is known to find globally optimal pass sequences.

Classical solutions rely on fixed schedules (`-O2`, `-O3`, `-Oz`) that represent manually tuned compromises over a broad class of programs. While effective in practice, these schedules cannot adapt to the structure of individual programs, leaving program-specific optimization opportunities unexploited. Iterative compilation methods search the pass-order space exhaustively but scale poorly; their cost is prohibitive for production compilers.

The recent rise of reinforcement learning (RL) has opened a new avenue for tackling the phase ordering problem. An RL agent can interact with the compiler, observe intermediate representations at each step, and learn a policy that selects passes sequentially—naturally fitting the sequential,

35 feedback-driven structure of pass ordering. To facilitate such learning-based approaches, Compiler-
36 Gym Cummins et al. [2022] was proposed as a standardized RL environment for compiler optimization.
37 It provides a unified interface across compiler infrastructures and benchmark datasets, enabling
38 RL agents to learn and evaluate optimization strategies in a reproducible, gym-compatible setting.

39 Despite the promise of RL for this task, recent work has moved toward large language model (LLM)-
40 based approaches Pan et al. [2026], which require orders of magnitude more compute and sacrifice
41 the interpretability and sample efficiency of classical actor-critic methods. This paper instead in-
42 vestigates how far *traditional* DRL can be pushed on the pass-ordering problem, using only the
43 56-dimensional Autophase IR representation Huang et al. [2019] and a discrete action space of 30
44 curated LLVM passes.

45 We make the following contributions:

- 46 • A state representation that combines online-normalized Autophase features with per-action
47 usage counts, providing the agent with both program structure and episode history.
- 48 • A truncation-aware GAE implementation that correctly handles time-limited episodes,
49 eliminating a systematic value-estimation bias present in standard PPO implementations.
- 50 • A parallel rollout infrastructure (8 workers) that reduces training wall-clock time by 7.6×
51 without degrading policy quality.
- 52 • An inference-time ensemble strategy that achieves state-of-the-art results on `cbench-v1`
53 and generalizes to six additional benchmark suites without any fine-tuning.

54 2 Related Work

55 2.1 Compiler Optimization as a Sequential Decision Problem

56 The problem of selecting a sequence of compiler optimization passes has long been recognized as a
57 combinatorial search problem. Early iterative-compilation systems such as MILEPOST Fursin et al.
58 [2011] treated pass ordering as a supervised or genetic-search problem, relying on offline models
59 that cannot adapt to individual program structure at inference time and are limited by the exponential
60 size of the pass-sequence space.

61 CompilerGym Cummins et al. [2022] formalizes compiler optimization as a Markov Decision Pro-
62 cess (MDP), providing a gym-compatible interface to LLVM optimization passes with reproducible
63 benchmarks and objective reward signals. Our work builds directly on this framework, using the
64 `llvm-v0` environment with Autophase observations and the `IrInstructionCount0z` reward.

65 2.2 Reinforcement Learning for Compiler Optimization

66 Huang et al. [2019] introduced Autophase, a 56-dimensional feature vector extracted
67 from LLVM IR, and demonstrated that a simple RL agent trained on this representation could sur-
68 pass fixed optimization levels on individual programs. Subsequent work proposed graph-based IR
69 representations (ProGraML) Cummins et al. [2021] for learning program structure, though their
70 training cost is substantially higher.

71 The recent Compiler-R1 Pan et al. [2026] applies LLMs with reinforcement fine-tuning to generate
72 pass sequences directly from IR source. While achieving strong results—an average 8.46% IR
73 instruction count reduction compared to `-Oz`—LLM-based methods require orders of magnitude
74 more compute, lack the sample efficiency of actor-critic methods, and are difficult to interpret at the
75 pass-selection level. Our work deliberately avoids LLMs and investigates how far traditional DRL
76 can be pushed with carefully designed state representations, training infrastructure, and inference-
77 time aggregation.

78 2.3 Proximal Policy Optimization

79 Proximal Policy Optimization (PPO) Schulman et al. [2017] is a widely adopted on-policy actor-
80 critic algorithm that constrains policy updates via a clipped surrogate objective. PPO has been
81 applied successfully to combinatorial action spaces in domains including chip placement Mirhoseini
82 et al. [2021], where discrete action sets of comparable size to our 30-pass action space are standard.

83 We adopt recurrent PPO with a gated recurrent unit (GRU) encoder to capture temporal dependencies
84 within the pass sequence of an episode.

85 2.4 Intrinsic Motivation

86 The Intrinsic Curiosity Module (ICM) Pathak et al. [2017] augments the extrinsic reward with a
87 prediction-error-based bonus that encourages exploration of novel states. We include ICM in our
88 architecture; however, our empirical analysis shows that the ICM signal decays sharply over training
89 and contributes minimally to final performance, consistent with findings in environments where the
90 extrinsic reward eventually dominates.

91 2.5 Ensemble Methods in RL

92 Ensemble methods that aggregate over independently trained policies have been studied in the con-
93 text of model uncertainty estimation Lakshminarayanan et al. [2017] and exploration Osband et al.
94 [2016]. Our work applies inference-time ensembling in the compiler domain: we train N indepen-
95 dent PPO checkpoints with varied hyperparameter settings and, for each benchmark program, keep
96 the best trajectory across all models and multiple stochastic rollouts.

97 3 Methodology

98 We frame compiler pass ordering as a finite-horizon MDP and solve it with a recurrent PPO agent
99 augmented with per-feature state normalization, a curated discrete action set, parallel rollout collec-
100 tion, and an inference-time ensemble.

101 3.1 MDP Formulation

102 **State.** At each step t , the state s_t consists of three components: (1) the 56-dimensional Autophase
103 vector $\phi(P_t)$ Huang et al. [2019] extracted from the current LLVM IR; (2) per-feature online nor-
104 malization via a Welford running-mean-variance estimator, $\hat{\phi}_i = (\phi_i - \mu_i)/(\sigma_i + \epsilon)$, which brings
105 heterogeneous Autophase values (ranging from 0 to over 500) to a comparable scale for the actor
106 and critic; and (3) a 30-dimensional per-action usage-count vector recording how many times each
107 pass has been applied during the current episode, enabling the policy to avoid redundant applications
108 and model pass-interaction history.

109 **Action.** The action set consists of 30 LLVM optimization passes (PASSES_V4) curated from the
110 actual pass schedule of -Oz. Experiments with the full 124-pass action space and with smaller
111 15-pass sets confirmed that 30 passes represent a sweet spot: large enough to span the effective
112 optimization frontier, small enough for the agent to learn useful ordering policies within a fixed
113 episode budget.

114 **Reward.** We use the IrInstructionCount0z reward provided by CompilerGym Cummins et al.
115 [2022], defined as the fractional reduction in IR instruction count relative to the -Oz baseline:

$$r_t = \frac{\text{IC}_{-Oz}(P) - \text{IC}(P_t)}{\text{IC}_{-Oz}(P)}. \quad (1)$$

116 A cumulative episode reward above 1.0 indicates that the agent’s pass sequence produces fewer
117 instructions than -Oz.

118 **Episode length.** Each episode runs for a maximum of $T = 45$ steps, after which the environment
119 is truncated by a TimeLimit wrapper. Extending to $T = 90$ steps degraded performance, confirming
120 that the binding constraint is the action set rather than the episode horizon.

121 3.2 Policy Architecture

122 We use a recurrent actor-critic network with a shared GRU backbone. The concatenated state rep-
123 resentation is projected through a two-layer MLP encoder (hidden sizes 256 and 128, ReLU activa-
124 tions) and fed into a single-layer GRU with hidden dimension 128. The GRU hidden state is shared

by the actor head (a linear projection to a 30-dimensional logit vector) and the critic head (a linear projection to a scalar value estimate). The recurrent architecture captures temporal dependencies between pass choices within an episode—a dependency that a feed-forward policy cannot model.

3.3 Truncation-Aware Generalized Advantage Estimation

Standard GAE Schulman et al. [2015] sets the bootstrap value to zero at terminal states. When an episode ends by time-limit truncation rather than by a true terminal transition, the correct target is $V(s_T)$, not zero. We fix this by maintaining a truncation buffer that distinguishes truncated transitions from truly terminal ones, bootstrapping $V(s_T)$ at truncation boundaries. This eliminates a systematic underestimation of long-horizon credit that biases value targets toward zero every 45 steps in uncorrected implementations.

3.4 Parallel Rollout Collection

We distribute rollout collection across $W = 8$ parallel workers. Each worker maintains its own CompilerGym Cummins et al. [2022] gRPC service instance and a local copy of current policy parameters. The main process broadcasts updated weights and normalization statistics at the start of each collection phase; workers collect trajectories independently and return them for aggregation and PPO Schulman et al. [2017] updates. Worker boundaries require the GRU hidden state to be reset at each worker’s episode start to prevent stale hidden states from corrupting value targets across segment boundaries. This infrastructure reduces wall-clock time per 300k training steps from approximately 3 hours to 47 minutes—a $7.6\times$ speedup.

3.5 Intrinsic Curiosity Module

To encourage early-training exploration of diverse pass sequences, we augment the extrinsic reward with an ICM Pathak et al. [2017] bonus, which uses the forward-model prediction error as a curiosity signal. Empirically, the ICM signal decays to negligible magnitude by mid-training as the policy converges; we retain it as an exploration regularizer for the initial training phase.

3.6 Inference-Time Ensemble

Given N independently trained checkpoints $\{\pi_1, \dots, \pi_N\}$ and a stochastic rollout budget of k per model, the ensemble selects for each program p :

$$\hat{R}(p) = \max_{i \in [N], j \in [k]} R(\tau_{i,j}(p)), \quad (2)$$

where $\tau_{i,j}(p)$ is the j -th rollout of model i on program p . We set $k = 8$ (best-of-8, bo8) and $N \in \{5, 6, 7\}$ across experiments. Each training run—with a distinct learning rate, entropy coefficient, or state-feature configuration—converges to a different attractor in policy space Lakshminarayanan et al. [2017], Osband et al. [2016]. The per-program maximum recovers the best-fitting attractor at inference time with no additional training cost, requiring only $N \times k$ environment evaluations per program.

3.7 Per-Suite Reward Normalization for Multi-Suite Training

When training jointly on programs from multiple benchmark suites, per-step rewards span $[-30, +5]$ across suites. In our preliminary joint-training attempt, this caused value-loss divergence at step 80k due to high-magnitude gradient spikes from the blas-v0 and opencv-v0 suites. We address this with a per-suite Welford running estimator that z-scores each per-step reward before it enters the PPO loss:

$$\tilde{r}_t^{(s)} = \text{clip}\left(\frac{r_t^{(s)} - \mu^{(s)}}{\sigma^{(s)} + \epsilon}, -5, +5\right), \quad (3)$$

where s indexes the benchmark suite. End-of-training per-suite reward standard deviations confirm the heterogeneity this normalization corrects: cbench-v1 $\sigma = 0.11$, blas-v0 $\sigma = 0.58$, opencv-v0 $\sigma = 4.44$ —a $42\times$ ratio between the least and most volatile suites. With normalization, value loss remains bounded in $[0.2, 0.8]$ throughout the full 300k-step run.

4 Experiments

4.1 Setup

Environment. We use CompilerGym Cummins et al. [2022] llvm-v0 with Autophase observations Huang et al. [2019] (56 dimensions), the 30-pass PASSES_V4 action set, and IrInstructionCount0z reward. Primary evaluation is on cbench-v1 (23 programs); cross-suite evaluation covers mibench-v1, blas-v0, npb-v0, chstone-v0, opencv-v0, and anghabench-v1.

Baselines. We compare against: (1) **Random policy**: actions sampled uniformly at random; (2) **PPO v3 baseline**: recurrent PPO trained for 200k steps on a single core with a 15-pass action set and raw (unnormalized) Autophase features; (3) compiler flags **-O2** and **-O3**; and (4) **-Oz** (implicit, since a mean reward of 1.0 means parity with -Oz).

Training. All models are trained for 300k environment steps on cbench-v1 using 8 parallel workers. We train seven models with varied learning rates (10^{-4} , 3×10^{-4}), entropy coefficients (0.01, 0.02), and state-feature ablations (with and without pass-sequence usage counts). Each training run completes in approximately 47 minutes on a single machine.

4.2 Main Results on cbench-v1

Table 1: Mean reward and number of programs beating -Oz on cbench-v1 (23 programs total) at best-of-8 (bo8) inference. Higher mean reward and more programs beaten is better.

Method	Mean Reward (bo8)	Programs > -Oz
Random policy	0.681	0 / 23
PPO v3 (baseline)	1.015	10 / 23
Best single model	1.044	17 / 23
Ensemble (ens5, $N=5$)	1.050	20 / 23
Ensemble (ens6, $N=6$)	1.049	20 / 23
Ensemble (ens7, $N=7$)	1.049	19 / 23

Table 1 shows results on cbench-v1. Compared to the v3 baseline (mean reward 1.015, 10/23 programs), our best ensemble reaches 1.050 mean reward and beats -Oz on 20 programs—a 3.5% improvement in mean reward and a doubling in the number of programs outperforming -Oz. The largest single gain came from the improved state representation (Section 3.1), which alone lifted greedy performance from 0.887 to 0.999 and bo8 performance from 1.015 to 1.042. Adding the inference-time ensemble contributed a further +0.006 in mean reward and raised the program-beating count from 17 to 20.

4.3 Cross-Suite Generalization

Table 2: Mean reward (bo8) across seven CompilerGym benchmark suites. All ensemble models were trained exclusively on cbench-v1; cross-suite evaluation is zero-shot. Best per-suite result in **bold**.

Method	cbench	mibench	blas	npb	chstone	opencv	angha
Random	0.681	—	—	—	—	—	—
PPO v3	1.015	—	—	—	—	—	—
Best single (v7)	1.042	1.028	0.447	4.45	1.10	0.20	0.44
ens5	1.050	1.030	0.967	4.520	1.112	2.869	1.079
ens6	1.049	1.032	0.948	4.551	1.111	2.754	1.079
ens7	1.049	1.030	0.976	4.528	1.113	2.985	1.090

Table 2 shows zero-shot transfer across six out-of-distribution suites. The ensemble consistently outperforms any single model. Notably, opencv-v0 improves from 0.20 (single model) to 2.985 (ens7),

193 a $14\times$ gain driven by the ensemble’s ability to select the best policy attractor per program Lakshmi-
194 narayanan et al. [2017].

195 4.4 Ablation Studies

196 **State representation.** Removing online normalization degrades bo8 performance on cbench-v1
197 from 1.042 to approximately 0.980, confirming that feature scale homogenization is critical. Remov-
198 ing per-action usage counts reduces greedy performance by approximately 0.015, as the agent loses
199 episode history and over-applies the same passes.

200 **Truncation-aware GAE.** The fix produces no isolated metric improvement on short episodes but
201 is a prerequisite for stable longer-horizon training: without it, value estimates are systematically
202 biased downward at every truncation boundary.

203 **Episode length.** Setting $T = 90$ reduces greedy performance to 0.937. The binding constraint is
204 the action set, not the time budget; extra steps with the same 30-pass set do not open new optimiza-
205 tion opportunities.

206 **Ensemble size.** Performance increases from a single model to ens5 and plateaus beyond that, con-
207 sistent with attractor diversity being saturated by 5–6 independent runs.

208 5 Conclusion

209 We have shown that traditional DRL—without any LLM component—can learn effective compiler
210 pass-ordering policies that substantially outperform the -Oz fixed schedule on the CompilerGym
211 benchmark Cummins et al. [2022]. Our three key contributions—improved state representation,
212 truncation-aware GAE Schulman et al. [2015], and an inference-time ensemble Lakshminarayanan
213 et al. [2017] with bo8 sampling—each provide independent, cumulative gains. The ensemble
214 generalizes zero-shot to six additional benchmark suites, demonstrating that policies trained on
215 cbench-v1 learn broadly applicable optimization knowledge.

216 Our analysis suggests that within the current recurrent PPO + Autophase design space, performance
217 is near saturation. The next phase of improvement likely requires structural changes such as graph IR
218 encoders (ProGraML Cummins et al. [2021]) or transformer-based sequence models over the pass
219 history. The inference-time ensemble, however, is a low-cost and immediately applicable strategy
220 for any existing RL-based compiler optimizer.

221 References

- 222 C. Cummins, Z. V. Fisches, T. Ben-Nun, T. Hoefler, M. F. OBoyle, and H. Leather. Programl:
223 A graph-based program representation for data flow analysis and compiler optimizations. In
224 *International Conference on Machine Learning*, pages 2244–2253. PMLR, 2021.
- 225 C. Cummins, B. Wasti, J. Guo, B. Cui, J. Ansel, S. Gomez, S. Jain, J. Liu, O. Teytaud, B. Steiner,
226 Y. Tian, and H. Leather. CompilerGym: Robust, Performant Compiler Optimization Environ-
227 ments for AI Research. In *CGO*, 2022.
- 228 G. Fursin, Y. Kashnikov, A. W. Memon, Z. Chamski, O. Temam, M. Namolaru, E. Yom-Tov,
229 B. Mendelson, A. Zaks, E. Courtois, et al. Milepost gcc: Machine learning enabled self-tuning
230 compiler. *International journal of parallel programming*, 39(3):296–327, 2011.
- 231 Q. Huang, A. Haj-Ali, W. Moses, J. Xiang, I. Stoica, K. Asanovic, and J. Wawrzyniek. Autophase:
232 Compiler phase-ordering for hls with deep reinforcement learning. In *2019 IEEE 27th Annual
233 International Symposium on Field-Programmable Custom Computing Machines (FCCM)*, pages
234 308–308, 2019. doi: 10.1109/FCCM.2019.00049.
- 235 B. Lakshminarayanan, A. Pritzel, and C. Blundell. Simple and scalable predictive uncertainty esti-
236 mation using deep ensembles. In *Advances in Neural Information Processing Systems (NeurIPS)*,
237 volume 30, 2017.

- 238 A. Mirhoseini, A. Goldie, M. Yazgan, J. W. Jiang, E. Songhori, S. Wang, Y.-J. Lee, E. Johnson,
239 O. Pathak, A. Nova, et al. A graph placement methodology for fast chip design. *Nature*, 594
240 (7862):207–212, 2021.
- 241 I. Osband, C. Blundell, A. Pritzel, and B. Van Roy. Deep exploration via bootstrapped dqn. In
242 *Advances in Neural Information Processing Systems (NeurIPS)*, volume 29, 2016.
- 243 H. Pan, H. Lin, H. Luo, Y. Liu, K. Yao, L. Zhang, M. Xing, and Y. Wu. Compiler-r1: Towards agentic
244 compiler auto-tuning with reinforcement learning. *Advances in Neural Information Processing*
245 *Systems*, 38:88289–88311, 2026.
- 246 D. Pathak, P. Agrawal, A. A. Efros, and T. Darrell. Curiosity-driven exploration by self-supervised
247 prediction. In *International conference on machine learning*, pages 2778–2787. PMLR, 2017.
- 248 J. Schulman, P. Moritz, S. Levine, M. Jordan, and P. Abbeel. High-dimensional continuous control
249 using generalized advantage estimation. *arXiv preprint arXiv:1506.02438*, 2015.
- 250 J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov. Proximal policy optimization
251 algorithms. *arXiv preprint arXiv:1707.06347*, 2017.